GREENTRACE 2.0

MASTER DEVELOPMENT PROMPT

Full specification for Antigravity or Claude coding agents

Objective: Build a real Tezos-based campus sustainability activity verification platform with wallet-signed submissions, organizer decisions, SHA-256 evidence fingerprints, dashboard, and public QR verification.

How to use: Paste this prompt into your coding agent. Ask it to inspect the workspace, verify current dependencies and testnet support, then implement one phase at a time. Do not request the entire application in one response.

MVP principle: A small end-to-end flow that genuinely works is better than many unfinished features.

1. Role and objective

- Act as a senior full-stack engineer, Tezos/SmartPy developer, security reviewer, UI/UX designer, and test engineer.
- Build GreenTrace 2.0: a campus sustainability activity verification web application. Deliver a working React frontend, FastAPI backend, SmartPy contract, wallet integration, evidence hashing, organizer approval, dashboard, QR verification, tests, and reproducible documentation. Do not deliver only a mockup or disconnected sample files.

2. Product and problem statement

- Students submit sustainability activities from their own Tezos wallets. Authorized organizers review evidence and approve or reject submissions using their own wallets. SHA-256 fingerprints help detect identical evidence. QR codes link to public verification of confirmed records.
- Blockchain proves the recorded transaction and contract state, not that a physical environmental activity definitely occurred. State this limitation clearly in the UI and demo.

3. Non-negotiable requirements

- Use Tezos, SmartPy, Taquito, and a compatible Beacon wallet integration. Student submissions and organizer decisions must be wallet-signed transactions.
- Exactly two authenticated roles: STUDENT and ORGANIZER. Public QR verification is read-only and needs no login.
- Do not replace wallet-based submission with college login or a backend relayer. These are optional future extensions only.
- Enforce authorization, unique activity IDs, valid state transitions, and self-approval prevention inside the smart contract—not just in frontend code.
- Hash actual evidence file bytes with SHA-256. Detect exact duplicates. Do not claim SHA-256 detects visually similar or resized images.
- Keep photographs, personal data, detailed descriptions not needed on-chain, and reviewer notes off-chain.
- Verify current official documentation for supported Tezos testnet, RPC, faucet, SmartPy tooling, Taquito APIs, and wallet compatibility. Do not assume Ghostnet is supported. Do not invent a contract address, transaction hash, confirmation, or test result.
- Never request or store a wallet password, private key, or recovery phrase. Never put private signing secrets in frontend code or commit secrets.

4. Technology stack

- Frontend: React, TypeScript, Vite, React Router, responsive CSS or Tailwind.
- Blockchain: SmartPy contract, Taquito, compatible Beacon wallet connection, verified Tezos RPC/testnet.
- Backend: Python, FastAPI, Pydantic validation, SQLite, protected evidence upload.
- Utilities: SHA-256, QR-code library, environment templates, automated tests, Git.
- Pin compatible dependency versions and follow current official documentation. Do not invent package names, APIs, commands, or obsolete wallet methods.

5. Roles and permissions

- **STUDENT:** connect/disconnect wallet; view connected address; submit activity; upload evidence; see hash and duplicate warning; sign submission; view own history and status; open details and QR page.
- Students cannot approve/reject, change submitter identity, edit other students' activities, self-approve, or grant organizer privileges.
- **ORGANIZER:** connect an authorized wallet; inspect pending records and evidence; review hash and duplicate warning; approve/reject; optionally enter rejection reason stored off-chain; sign decision; see history and totals.
- Organizers cannot approve their own submissions, act from unauthorized wallets, overwrite a decision, or change records through the database instead of contract calls.
- Use a documented initial administrator/deployer to authorize organizers. Do not create a third authenticated admin portal unless essential. Document student eligibility: an arbitrary wallet does not prove college enrollment.

6. Wallet lifecycle

- Connect wallet → obtain actual address from provider → verify selected network → resolve role using contract permissions/policy → request wallet signature for protected action → submit operation → wait for confirmation → show actual operation hash and confirmed state.
- Handle missing wallet, connection/signature rejection, account change, disconnect, unsupported network, insufficient test funds, RPC errors, contract errors, pending state and confirmation timeout.
- Never accept a manually typed address as identity. Never claim success before confirmation.

7. Student submission inputs

- Required category: Recycling, Tree planting, Energy conservation, Water conservation, Waste collection, Other.
- Required positive quantity and compatible unit: kg for recycling/waste; saplings for planting; Wh/kWh for energy; litres for water; Other must specify a unit.
- Required short description and evidence image for the MVP. Optional reported activity date must be distinguished from contract timestamp.
- Generated values: unique activity ID, SHA-256 hash, contract timestamp, SUBMITTED status and actual transaction hash.
- Validate quantity, category/unit pairing, file size/type/content, required description, connected wallet, correct network, duplicate hash, and duplicate ID. Disable repeat submissions while upload/transaction is in progress.

8. SHA-256 and duplicate detection

- Hash actual file bytes and encode lowercase hexadecimal consistently. Backend independently verifies the hash. Check duplicate hash before submission and block normal exact-duplicate submissions or document an explicit exception policy.
- Use database uniqueness constraints and safe handling of concurrent/repeated requests. If duplicate prevention must be guaranteed on-chain, enforce it in contract storage too.
- Explain that recompressed, resized, or modified images may produce a different SHA-256 hash. Perceptual hashing is future work.

9. SmartPy contract and data model

- Each activity record: activity_id, evidence_hash, category, quantity, unit, status, submitter_address, approver_address, submitted_at, approved_at, and optional superseding-record reference.
- Use a suitable map/big-map keyed by activity ID. Avoid unnecessary storage; never store photographs or personal information on-chain.
- Statuses: SUBMITTED, APPROVED, REJECTED. Add other states only if justified and tested.
- submit_activity: derive submitter from Tezos sender; validate unique ID, category, quantity, unit, eligibility and duplicate policy; set SUBMITTED and contract timestamp.
- approve_activity: authorized organizer only; record must exist and be SUBMITTED; reject self-approval; derive approver from sender; record timestamp and set APPROVED.
- reject_activity: authorized organizer only; record must exist and be SUBMITTED; derive approver from sender; set REJECTED without overwriting prior decisions. Keep detailed rejection reason off-chain.
- add_organizer/remove_organizer: administrator only. Use explicit assertions for permissions and transitions. Compile and test with current SmartPy tooling before deployment.

10. On-chain vs off-chain

- On-chain: activity ID, evidence hash, category, quantity, unit, status, submitter and approver public addresses, relevant timestamps, and correction reference if needed.
- Off-chain: names/emails if collected, original images, file paths, detailed descriptions, rejection reasons, reviewer notes, preferences and operational logs.
- Blockchain is authoritative for confirmed state. SQLite is a cache/application database, not proof of chain state. Reconcile cached records with contract state.

11. Backend and database

- FastAPI endpoints may include GET /health; POST /api/evidence; GET /api/evidence/{hash}/duplicate-check; GET /api/activities; GET /api/activities/{id}; GET /api/activities/{id}/history; GET /api/public/activities/{id}. Adjust to actual architecture and document all routes.
- Validate uploads, calculate server-side hash, detect duplicates, store protected files, maintain off-chain metadata, store transaction references, provide public-safe records, and reconcile chain/cache state.
- SQLite activities fields may include activity_id, submitter_address, category, quantity, unit, description, evidence_hash, evidence_path, status_cache, submission_tx_hash, approval_tx_hash, submitted_at, approved_at, rejection_reason. Add organizer and audit tables as useful.
- Use indexes/unique constraints and safe retries for cases where a transaction confirms but database update fails. Backend must not sign or relay student/organizer contract operations.

12. Required pages

- Landing page with project explanation and Connect Wallet.
- Wallet connection/status component with address, network indicator, disconnect and errors.
- Student dashboard with counts, recent history and submit action.
- Submission form with validation, upload, hash, duplicate warning and transaction states.
- Activity details with ID, category, quantity/unit, status, addresses, hash, timestamps and transaction references.
- Organizer dashboard with pending queue, evidence review, duplicate warning and approve/reject.
- Public QR verification page with public record, network, contract address and explorer links. Do not expose private photos, email, or unnecessary personal data.

13. QR verification and UI

- Generate a QR code linking to a public route with activity ID. The page must query the correct contract/network and verify the actual chain record before showing a verified state.
- If RPC or contract lookup fails, display verification unavailable. Never fabricate a VERIFIED badge. Clearly distinguish cached metadata from on-chain verified state.
- Use accessible responsive layouts, clear status badges, loading/empty/error states, decision confirmation, and meaningful transaction progress. Sample data must be labelled as sample and never presented as real chain data.

14. Security and failure handling

- Implement contract access control, self-approval prevention, duplicate ID/hash policy, safe file uploads, restricted evidence access, server validation, environment configuration, CORS, safe logging, transaction failure states and reconciliation.
- Never trust frontend role flags; hard-code credentials; expose secrets; publish private evidence; treat pending operations as confirmed; or use a database status as proof of chain state.

15. Tests

- SmartPy tests: valid submission, invalid quantity/category/unit, duplicate ID/hash, unauthorized approval/rejection, self-approval, repeated decisions, decision on non-pending activity, unauthorized organizer management.
- Frontend/backend tests: form validation, upload validation, duplicate requests, transaction error handling, QR lookup, privacy, API errors, cache reconciliation.
- Run frontend build/type checks, backend tests, SmartPy compile/tests and live testnet checks where possible. Label live-network tests NOT TESTED if prerequisites are unavailable.

16. Implementation phases

- Phase 1: scaffold and responsive frontend. Phase 2: wallet connection and role resolution. Phase 3: SmartPy contract/tests and deployment preparation. Phase 4: student wallet-signed submission. Phase 5: upload, SHA-256 and duplicates. Phase 6: organizer wallet-signed decision. Phase 7: SQLite/backend reconciliation. Phase 8: dashboards/history. Phase 9: QR verification. Phase 10: security review, end-to-end tests and documentation.

- Each phase must list files changed, complete code, commands, tests run, actual results, errors fixed, and manual setup required. Preserve working code and implement one phase at a time.

17. Documentation and definition of done

- Provide README.md, ARCHITECTURE.md, TESTING.md and DEMO_GUIDE.md. Include Windows PowerShell commands, prerequisites, dependency versions, environment variables, wallet setup, testnet funding, deployment, organizer authorization, run commands and troubleshooting.
- Done means: student connects wallet; valid submission is signed and confirmed; duplicate evidence is detected; authorized organizer signs approval/rejection; contract blocks unauthorized/self-approval; dashboard shows actual data; QR page verifies actual chain record; private evidence is protected; tests and setup are reproducible.
- Begin by inspecting the workspace and verifying current dependency/testnet support. Present a plan, then implement Phase 1 only. Do not generate the whole application in one enormous response.